



A Batch Solver for Initial Value Problems on GPUs

Ben Longaker¹

April 30, 2026

¹The University of Texas at Austin, Oden Institute for Computational Engineering and Sciences

Initial Value Problems (IVPs)

$$\begin{cases} \dot{\mathbf{y}} = f(t, \mathbf{y}; \mathbf{p}), \\ \mathbf{y}(t_0) = \mathbf{y}_0, \end{cases}$$

$\mathbf{y}(t)$: state vector, $\mathbf{p}$: vector of fixed parameters. Goal: given many different $\mathbf{y}_0, \mathbf{p}$, compute $\mathbf{y}(t_f)$ for each system.

- Important application: astrochemistry. We need to solve a large IVP for each spatial grid cell, for each (outer loop) time step. This is a major computational bottleneck in these simulations; it is necessary to have surrogate/fast batched solver.

The simplest time-stepping algorithm is **forward Euler**:

$$\mathbf{y}_{k+1} = \mathbf{y}_k + \Delta t \cdot f(t_k, \mathbf{y}_k; \mathbf{p}) \quad (\text{FE})$$

$$\mathbf{y}_{k+1} = \mathbf{y}_k + \Delta t \cdot f(t_k, \mathbf{y}_{k+1}; \mathbf{p}) \quad (\text{BE})$$

- This is a **first-order method**: as $\Delta t \rightarrow 0$, the error of the solution decreases at a linear rate.
- Extremely simple to implement, assuming the time step is fixed.
- Extremely easy to implement and parallelize.

Adaptive Time-Stepping

In most applications, we would like to change the time step to match the dynamics. This is done by considering two time-steppers of different order and comparing the difference. For example: **RK23** (Bogacki, Shampine 1997):

$$k_1 = f(t_k, \mathbf{y}_k)$$

$$k_2 = f\left(t_k + \frac{\Delta t}{2}, \mathbf{y}_k + \frac{\Delta t}{2}k_1\right)$$

$$k_3 = f\left(t_k + \frac{3\Delta t}{4}, \mathbf{y}_k + \frac{3\Delta t}{4}k_2\right)$$

$$\mathbf{y}_{k+1}^{(3)} = \mathbf{y}_k + \frac{\Delta t}{9}(2k_1 + 3k_2 + 4k_3)$$

$$t_{k+1} = t_k + \Delta t$$

$$k_4 = f\left(t_{k+1}, \mathbf{y}_{k+1}^{(3)}\right)$$

$$e_{k+1} = \frac{\Delta t}{72}(-5k_1 + 6k_2 + 8k_3 - 9k_4).$$

- If $\|e_{k+1}\| > \epsilon_{\text{tol}}$, we shrink the time step and recompute. In either case we compute a new time step to use for the next:

$$\Delta t^{(k+1)} = \Delta t^{(k)} * \left(\frac{\epsilon_{\text{tol}}}{\|e_{k+1}\|} \right)^{0.2}.$$

This is more difficult to parallelize on a GPU since some threads may desynchronize.

- If each ODE is solved on a single thread, some ODEs may reach t_f faster than others. Hence many threads will remain idle while the others finish.

Parallel Algorithm

Parallel Algorithm

Idea: solve one ODE per thread.

- For forward Euler, every ODE reaches the final time after the same number of iterations; no desync.

The work-depth model for this algorithm is extremely simple due to the embarrassingly parallel nature:

- **Work (W):** Let N_{ODE} be the number of ODEs we solve, d the dimension (# state variables) of each ODE, T the number of time steps, and q_{RHS} the cost of evaluating a right-hand side. Then:

$$W = O(N_{\text{ODE}} \cdot d \cdot T \cdot q_{\text{RHS}})$$

- **Depth (D):** Since each ODE is solved independently, the depth is the cost of solving a single ODE:

$$D = O(d \cdot T \cdot q_{\text{RHS}})$$

Implementation

- Use Python **CuPy** interface, with **Raw Kernels** for the solvers.
- To include custom RHS, embed function code directly into the CUDA kernel (written in C) before compiling.
 - Best of both worlds: **Python** for user interface, low-level **C** makes the solver extremely fast.
- **Forward Euler**: each thread finishes at exactly the same time. Zero thread divergence.
- **RK23**: some threads finish computing before others. But during computing, each thread does exactly the same operations.
 - For now, assume this is not an issue.

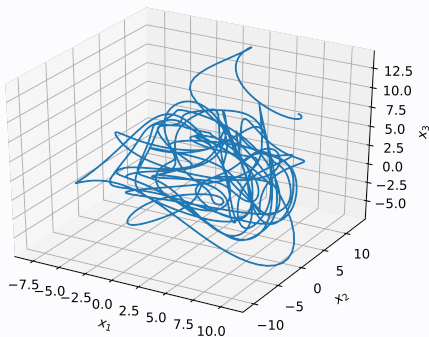
Results

Test Problem: Lorenz96

We test the Lorenz96 ODE:

$$\dot{x}_i = (x_{i+1} - x_{i-2})x_{i-1} - x_i + F,$$

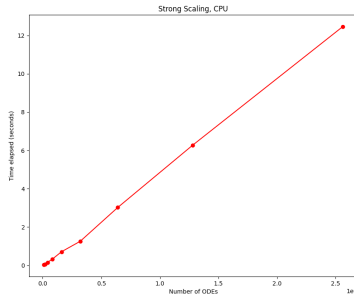
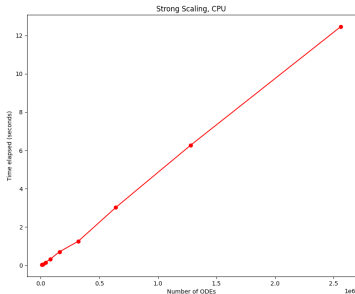
and take the initial condition randomly generated in the cube $[-5, 5]^d$, and randomly generate $F \in [1, 5]$.



Comparing Time-Stepping

Compare two different compute kernels:

- `forward_euler_kernel`: on one thread, takes a single time step. Measure time for steps taken until completion.
- `forward_euler_kernel_multistep`: on one thread, runs a single IVP to completion. Measure time for a single launch



Comparison to Serial Code

N_{ode} ($\times 10^6$)	Parallel Time (s)	Serial Time (s)	Speedup

Next Steps

- Add load balancing.
 - Idea: run for some number of time steps. Then rearrange data on the GPU so that faster ODEs relaunch on the same thread.
- Add implicit time stepping.
 - Solving nonlinear system of equations for each time step. Challenging to do per thread if d is large.
- Thank you!